

Rapport d'Étude et Réalisation

Semestre 3 – 2024/2025

Projet : Carte Puissance industrialisé

Professeur encadrant : Yves GUINAND



Sommaire

Remerciements	4
1. Présentation du Projet	5
1.1. Schéma synoptique de la carte puissance	5
2. Objectifs et tâches à accomplir	6
3. Actions menées dans le projet.....	7
3.1. Prise en main du STM32.....	7
3.2. Test du bus CAN.....	10
3.3. Mesures des tensions et courants sur l'ancienne carte	13
3.4. Conception de la nouvelle carte : Schématique.....	14
3.4.1. Schéma de l'Alimentation électronique et son circuit de mesure	14
3.4.2. Schéma du microcontrôleur et du connecteur de programmation.....	15
3.4.3. Schéma des alimentations protégé et ses circuits de mesure	16
3.4.4. Schéma du driver CAN.....	17
3.5. Conception du PCB	18
3.6. Test et Programmation de la nouvelle carte	20
3.6.1. Test et programmation des capteurs	20
3.6.2. Étalonnage des capteurs.....	22
3.6.3. Programmation de l'interruption sur l'arrêt d'urgence	23
3.6.4. Détection d'un court-circuit.....	24
3.6.5. Programmation du BUS CAN.....	24
4. Résultats obtenus.....	28
Conclusion	29

Liste des figures

Figure 1 : Schéma synoptique de la carte puissance.....	5
Figure 2 : Pinout du STM32 NUCLEO-F303K8.....	7
Figure 3 : Légende des pins du STM32.....	8
Figure 4 : câblage pour le test des Entrées/Sorties	8
Figure 5 : Câblage pour le test du bus CAN	10
Figure 6 : Messages envoyé et reçu par l'USB2CAN et moniteur série	12
Figure 7 : Câblage de la mesure de tension et courant sur l'ancienne carte	13
Figure 8 : Schéma de l'alimentation électronique et son circuit de mesure	14
Figure 9 : Protection aux polarités inverse.....	14
Figure 10 : Schéma du microcontrôleur et du connecteur de programmation	15
Figure 11 : Bloc d'alimentation 1 et résistances de pull-up	16
Figure 12 : Schéma du driver CAN	17
Figure 13 : Placement recommandé de l'INA236.....	18
Figure 14 : Modèle 3D de la carte envoyé en production.....	19
Figure 15 : Câblage du test de la carte.....	20
Figure 16 : Câblage pour la procédure d'étalonnage	22
Figure 17 : Fonctionnement du codage/décodage d'un flottant dans le bus CAN	25
Figure 18 : Capture d'écran montrant le filtrage des ID.....	26
Figure 19 : Traitement des trames CAN reçu	27
Figure 20 : Affichage des valeurs lue par les capteurs.....	28
Figure 21 : Carte industrialisé à côté de l'ancienne carte.....	28

Liste des tableaux

Tableau 1 : Planning des séances d'E&R.....	6
Tableau 2 : Adresses de l'INA236	16
Tableau 3 : Étalonnage des capteurs	23
Tableau 4 : Trames CAN envoyées et reçu	24
Tableau 5 : Comparaison de la taille de la carte industrialisée contre l'ancienne carte	28

Remerciements

Avant de commencer je vais remercier mon professeur encadrant M.Guinand qui m'as guidé durant tout le projet et aidé à résoudre certains problèmes, M.Ardiller qui a soudé la carte et enfin le reste du corps enseignant qui m'ont transmis leurs connaissances en électronique et en programmation.

1. Présentation du Projet

Pour la Coupe de France de robotique de 2025, les membres du CRAC se sont répartis des tâches à réaliser pour la compétition. Pour l'édition 2025, on doit concevoir un robot et des PAMI pour préparer une salle de concert.

J'ai décidé de faire l'industrialisation de la Carte Puissance à partir de la carte qui avait été faite par Kaushik l'an dernier. Le rôle de la carte est d'alimenter le robot, protéger les circuits du robot et enfin surveiller les alimentations du robot. Durant ce projet il faut concevoir la version finale de la carte pour réduire la taille de la carte et avoir une carte plus performante, cette carte sera utilisée pour les prochaines années.

1.1. Schéma synoptique de la carte puissance

Après avoir effectué les essais sur l'ancienne carte pour comprendre son fonctionnement, j'ai conçu le schéma synoptique suivant de la nouvelle carte avec les changements de composants :

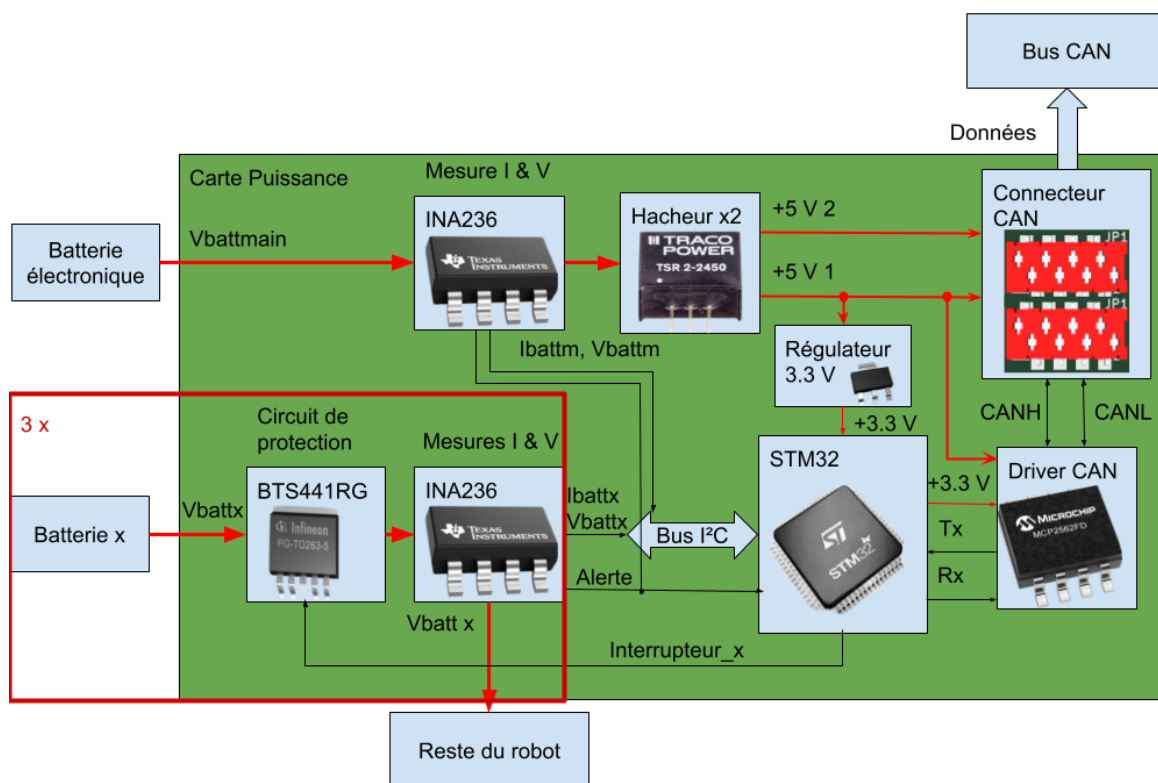


Figure 1 : Schéma synoptique de la carte puissance

Cette carte est alimentée par une batterie dédiée à l'électronique du robot, la tension et le courant de la batterie sont mesurés par l'INA236 qui donne les informations avec une liaison I2C, un capteur de tension et courant. La tension de la batterie est abaissée deux fois,

une première fois par des TRACO pour avoir du 5V que l'on utilise pour alimenter les cartes connectées au bus CAN et une deuxième fois pour alimenter le STM32F303K8 en 3.3 V.

La carte possède trois bornes d'alimentations et sortie, en sortie on a une alimentation protégée que l'on peut décider de couper ou allumer à l'aide du bus CAN. On y branche une batterie, un circuit de protection et interrupteur programmable (Le BTS441RG) protège la sortie des courts-circuits, polarité inverse (si on branche la batterie à l'envers), surcharge etc. Et l'INA236 mesure la tension et courant de la batterie et les envoie au microcontrôleur.

Un driver CAN permet la communication avec les appareils branché au bus CAN, la carte envoie en continu les tensions et courant, informe les appareils si l'interrupteur arrêt d'urgence est actionné, si un court-circuit est détecté et sur quelle voie et on peut commander les interrupteurs depuis le bus CAN.

2. Objectifs et tâches à accomplir

Pour ce projet les objectifs et tâches sont les suivantes :

- Comprendre comment l'ancienne carte fonctionne (test des fonctionnalités de la carte)
- Concevoir la carte
 - o Changer les circuits de mesures
 - o Intégrer la puce du microcontrôleur directement sur la carte
- Tester le bon fonctionnement de la carte
- Programmer les fonctionnalités de la carte

Le planning des séances d'E&R a été le suivant :

Tableau 1 : Planning des séances d'E&R

Séances	Tâches
1	Visite du CRAC et Répartition des projets
2	Test du STM32 sur Arduino
3	Test de l'ancienne carte : CAN
4	Test de l'ancienne carte : CAN et Mesure de tension
5	Test de l'ancienne carte : Mesure de courant
6	Conception de la nouvelle carte : Schématique
7	Conception de la nouvelle carte : PCB
8	Liste des fonctionnalités à programmer, début du programme

9	Début des tests de la nouvelle carte
10	Programmation : Mesure de tension et courant
11	Programmation : ARU et envoi des informations dans le bus CAN
12	Programmation : Test de la broche alert
13	Programmation : Filtre des ID du bus CAN et commande des interrupteurs via le bus CAN

3. Actions menées dans le projet

3.1. Prise en main du STM32

Pour ce projet le microcontrôleur utilisé est le STM32F303K8, l'on peut le programmer en C++ sur le Framework Mbed, cependant ce Framework sera arrêté d'être mis à jour en 2026, il a été décidé que l'on migre dans le Framework Arduino.

Le pinout du STM32 est le suivant :

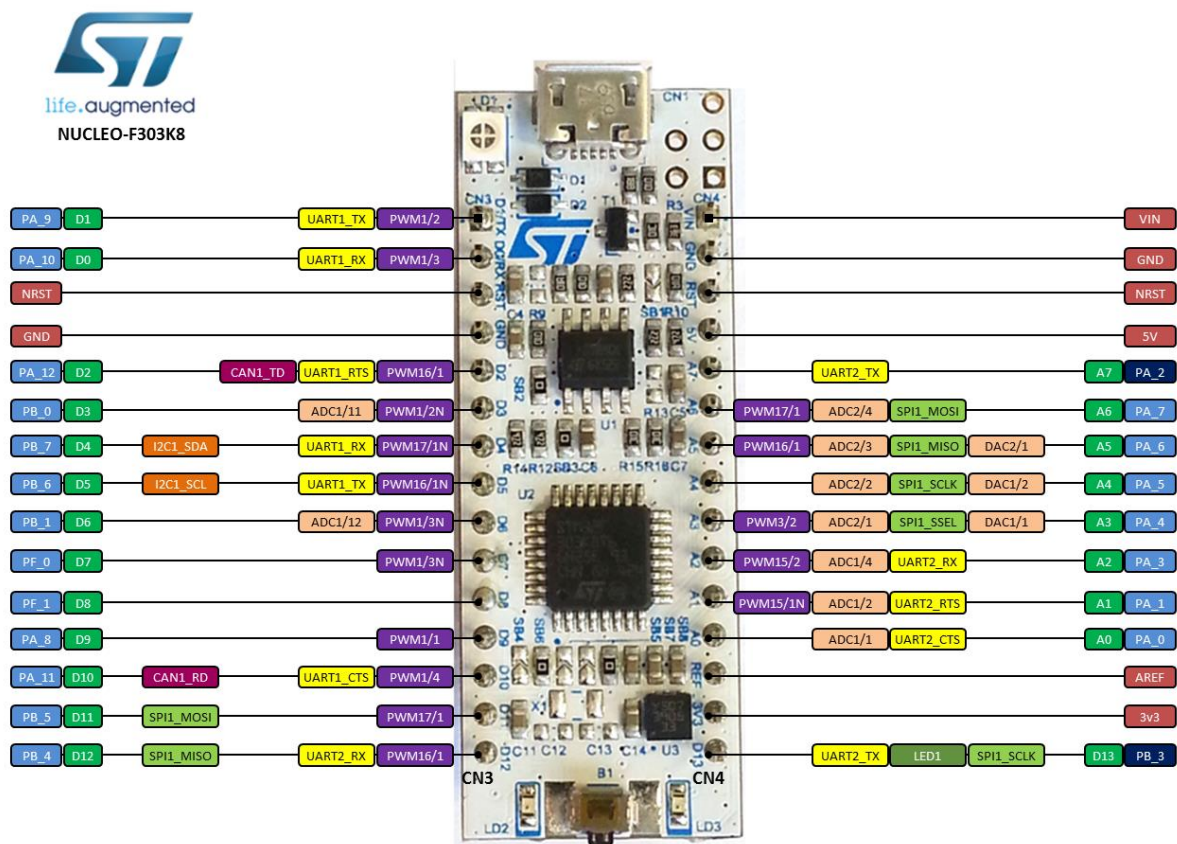


Figure 2 : Pinout du STM32 NUCLEO-F303K8

Et la légende des pins :

Labels usable in code

PX_Y	MCU pin without conflict	XXX	Arduino connector names (A0, D1, ...)
PX_Y	MCU pin connected to other components See PeripheralPins.c (link below) for more information	XXX	LEDs and Buttons (LED_1, USER_BUTTON, ...)

Labels not usable in code (for information only)

XXX	Serial pins (USART/UART)	XXX	AnalogIn (ADC) and AnalogOut pins (DAC)
XXX	SPI pins	XXX	CAN pins
XXX	I2C pins	XXX	Power and control pins (3V3, GND, RESET, ...)
XXX	PWMOut pins (TIMER n/c[N]) n = Timer number c = Channel N = Inverted channel		

Figure 3 : Légende des pins du STM32

Attention les broches PA5 et PA6 ne peuvent qu'être utilisés en entrée analogique, pour les utiliser en sortie TOR et utiliser d'autres fonctions (SPI, PWM, etc.) il faudra dessolder les composants SB16 et SB18.

J'ai commencé par créer un projet sur PlatformIO, pour notre microcontrôleur, il faut choisir la board : **ST Nucleo F303K8**, sous le Framework Arduino. Après avoir essayé de téléverser un programme vide sur le microcontrôleur, j'ai remarqué que le protocole pour téléverser le programme par défaut (stlink) ne fonctionnait pas et que le seul qui fonctionne est mbed. Dans le platformio.ini, il faut changer l'upload_protocol :

```
upload_protocol = mbed
```

Cette fois le programme a été téléversé dans le microcontrôleur. J'ai ensuite testé les entrées et sorties du microcontrôleur. Les câblages pour les tests sont les suivantes.

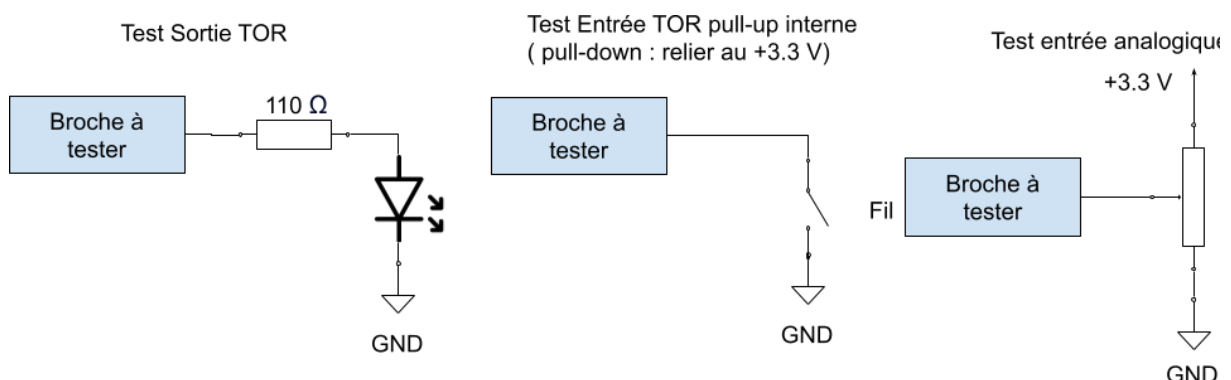


Figure 4 : câblage pour le test des Entrées/Sorties

Pour les entrées et sortie TOR, on les initialise avec : **pinMode(PIN, Mode)** ; on y précise le nom du pin, et le mode de fonctionnement (Entrée pull-up/down interne ou non, sortie) :

Pour la lecture des entrées on utilise la fonction **digitalRead(PIN)** ; et on stocke la valeur dans une variable et on écrit sur les sorties avec : **digitalWrite(PIN, HIGH/LOW)** ; (LOW : 0 et HIGH : 1).

Pour une entrée analogique, aucune initialisation à faire, et on utilise : **analogRead(PIN)** ; qui retourne la valeur lue.

Pour une sortie PWM, on initialise la broche en sortie (comme pour le TOR) et on utilise **analogWrite(Valeur)** ; pour générer une PWM en sortie et la valeur est compris entre 0-255 (0%-100%).

3.2. Test du bus CAN

Après avoir pris en main le microcontrôleur, j'ai testé le bus CAN de l'ancienne carte avec l'USB2CAN-C pour observer les trames sur l'ordinateur et la bibliothèque STM32_CAN que l'on peut trouver sur GitHub (https://github.com/pazi88/STM32_CAN) et sur PlatformIO.

Le câblage pour le test du bus CAN est le suivant (La carte est alimenté en 7,2 V) :

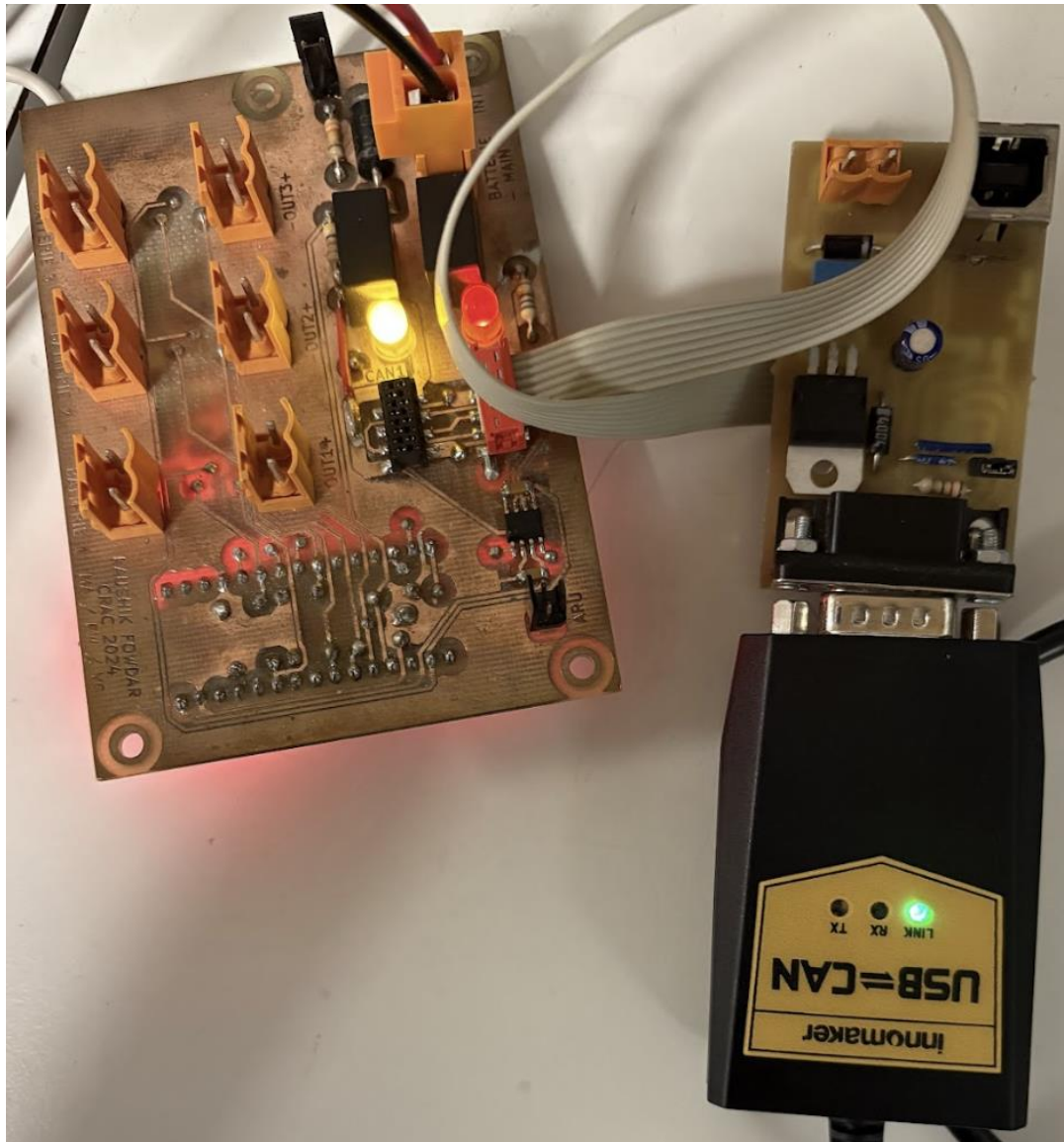


Figure 5 : Câblage pour le test du bus CAN

Le logiciel pour utiliser le USB2CAN-C et son manuel d'utilisateur se trouve sur GitHub :

<https://github.com/INNOMAKER/usb2can/blob/master/Document%EF%BC%88read%20me%20first%EF%BC%89/USB2CAN%20UserManual%20v.1.8.pdf>

Après avoir ajouté la bibliothèque STM32_CAN, il faut ajouter le HAL (Hardware Abstraction Layer) dans le .init pour que la bibliothèque fonctionne :

```
build_flags = -DHAL_CAN_MODULE_ENABLED
```

Pour pouvoir utiliser le bus CAN il faut déclarer un objet de la classe **STM32_CAN**, et un ou plusieurs messages de la structure **CAN_message_t** :

```
STM32_CAN Can1(CAN1, DEF); //Utilise les broches PA11/12 pour CAN1.
static CAN_message_t CAN_TX_msg;
static CAN_message_t CAN_RX_msg;
```

Ensuite on initialise le bus CAN dans le setup avec le Baud-rate que l'on veut utiliser :

```
Can1.begin();
Can1.setBaudRate(1000000); // 1MBaud/s
```

Pour pouvoir envoyer une trame :

```
CAN_TX_msg.id = (0x1A5); // donne l'id de la trame
CAN_TX_msg.len = 8; // Longueur de la trame ici 8 octet
CAN_TX_msg.buf[0] = 0x03; // données
CAN_TX_msg.buf[1] = 0x41;
CAN_TX_msg.buf[2] = 0x11;
CAN_TX_msg.buf[3] = Counter;
CAN_TX_msg.buf[4] = 0x00;
CAN_TX_msg.buf[5] = 0x00;
CAN_TX_msg.buf[6] = 0x00;
CAN_TX_msg.buf[7] = 0x00;
Counter++;
Can1.write(CAN_TX_msg); // envoi de la trame
```

Pour pouvoir recevoir une trame :

```
if(Can1.read(CAN_RX_msg)){ // regarde si une trame a été reçu
    //affichage de l'id et du message
    Serial.printf("ID %x\n",CAN_RX_msg.id);
    Serial.printf("msg :");
    for(int i; i < 8; i++){
        Serial.printf("%x |", CAN_RX_msg.buf[i]);
    }
    Serial.printf("\n");
}
```

Après avoir téléversé le programme de test du bus CAN sur la carte, on configure le logiciel pour qu'il reçoit correctement les trames de notre carte :

- Pour détecter l'appareil : Scan device
- On fixe le bon baud-rate, dans notre cas : 1 Mbaud/s
- Format : Standard
- Pour envoyer un message avec l'USB2CAN, on met l'id, et les données

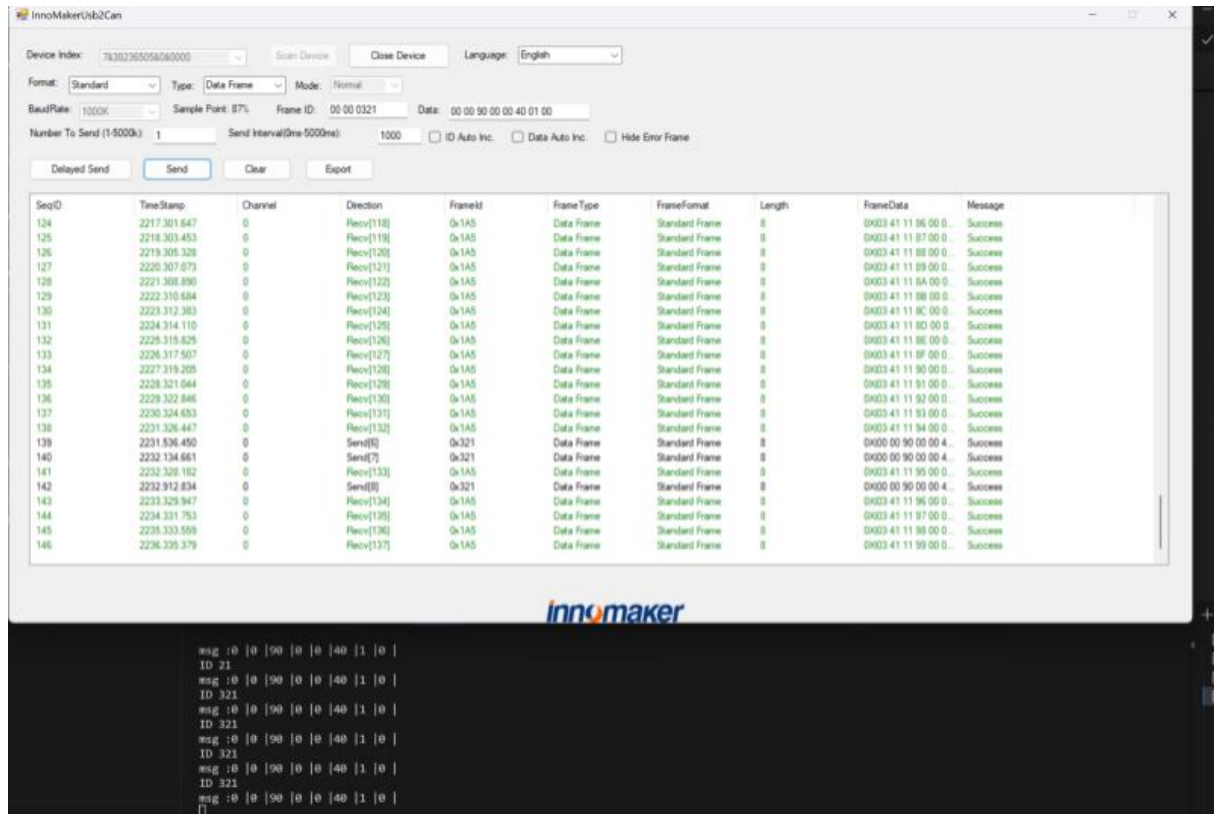


Figure 6 : Messages envoyé et reçu par l'USB2CAN et moniteur série

En noir les trames envoyées par l'USB2CAN, et en vert les trames de la carte, et en dessous on a le moniteur série qui affiche les trames que la carte reçoit.

3.3. Mesures des tensions et courants sur l'ancienne carte

Le câblage pour les mesures de tension et courant sur l'ancienne carte est le suivant, l'alimentation pour les voies des batteries est limitée en courant, pour les mesures il faut fermer le contact des interrupteurs que le courant puisse passer :

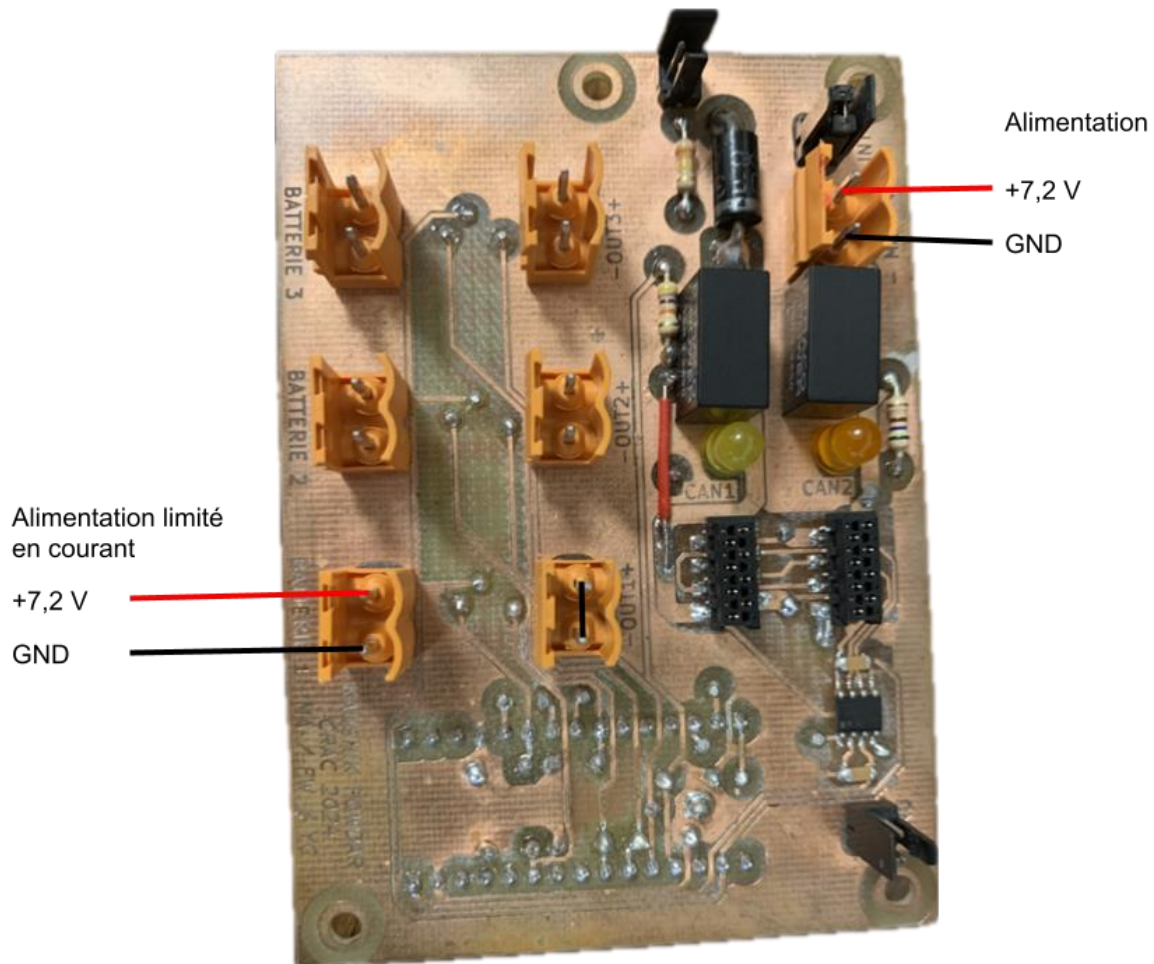


Figure 7 : Câblage de la mesure de tension et courant sur l'ancienne carte

Pour la mesure de tension, la carte utilise un pont diviseur de tension pour abaisser la tension de la batterie pour la mesurer avec le microcontrôleur. On utilise la fonction **analogRead** pour lire la tension. On peut retrouver la tension de la batterie : $V_{batt} = 3.3 \cdot \frac{N}{1023 \cdot G}$, avec G le gain du pont diviseur de tension.

Pour la mesure de courant, la carte utilise un capteur à effet hall (ACS711KLCTR-25AB-T), le gain de ce capteur est 55 mV/A. On utilise **analogRead** pour lire le courant, la valeur lue pour 0 A est de 511-512 (la moitié du maximal) il faut donc soustraire cette valeur par 512 et donc pour retrouver le courant : $I_{Batt} = 3.3 \cdot \frac{N-512}{1023 \cdot G}$ avec G le gain du capteur.

3.4. Conception de la nouvelle carte : Schématique

Après avoir fini les tests de l'ancienne carte, j'ai commencé la conception de la nouvelle carte en commençant par la conception du schématique. Les changements par rapport à l'ancienne carte sont les suivants :

- Module du STM32 remplacé par la puce
- Mesures de tension et courant réunit en 1 capteur l'INA236
- Nouveau circuit de protection aux polarité inverse sur l'alimentation électronique
- Passage aux LED CMS

3.4.1. Schéma de l'Alimentation électronique et son circuit de mesure

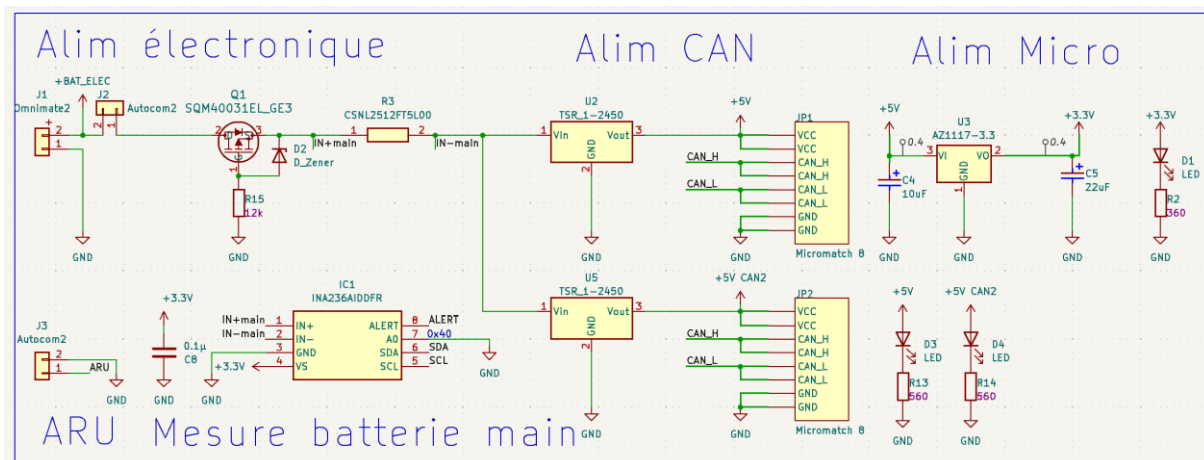


Figure 8 : Schéma de l'alimentation électronique et son circuit de mesure

On a un jumper pour l'alimentation électronique, puis un circuit pour protéger la carte des polarités inverse qui utilise un PMOS, le PMOS est passant lorsque le Vgs (Gate-Source Voltage) est plus grand que sa tension de seuil ($V_{gs} < -2.0\text{ V}$), ce qui arrive que quand l'on branche l'alimentation dans le bon sens et la diode zener permet de ne pas dépasser la tension maximale que le PMOS peut supporter ($\pm 20\text{ V}$).

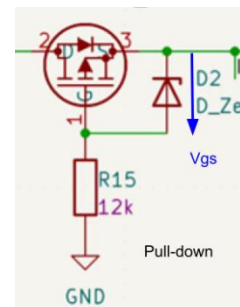


Figure 9 : Protection aux polarités inverse

Des convertisseurs à découpage les Traco TSR 2-2450 (ceux sur le schéma n'ont pas le bon symbole et empreinte) permettent d'abaisser la tension pour avoir du 5 V que l'on peut utiliser pour alimenter les cartes connectées au bus CAN.

Un régulateur linéaire permet de convertir le 5 V en 3,3 V pour alimenter le microcontrôleur.

Des LED permettent de vérifier si le CAN et microcontrôleur sont alimentés.

L'interrupteur d'arrêt d'urgence est câblé en pull-up, avec le pull-up interne du microcontrôleur.

L'INA236 permet de mesurer la tension et le courant de l'alimentation à l'aide de sa résistance de shunt, il envoie les informations avec le bus I2C.

3.4.2. Schéma du microcontrôleur et du connecteur de programmation

Le symbole du microcontrôleur et du connecteur pour le programmer a été récupéré sur un autre projet, de même pour le câblage, ce qui change est :

- SCL et SDA (sur PB6 et PB7) pour la liaison I2C avec les INA236
- Les signaux pour commander les interrupteurs des voies 1-3 (PowerSW1-3)
- Le signal Alert de l'INA236
- Le signal de l'interrupteur d'arrêt d'urgence en pull-up

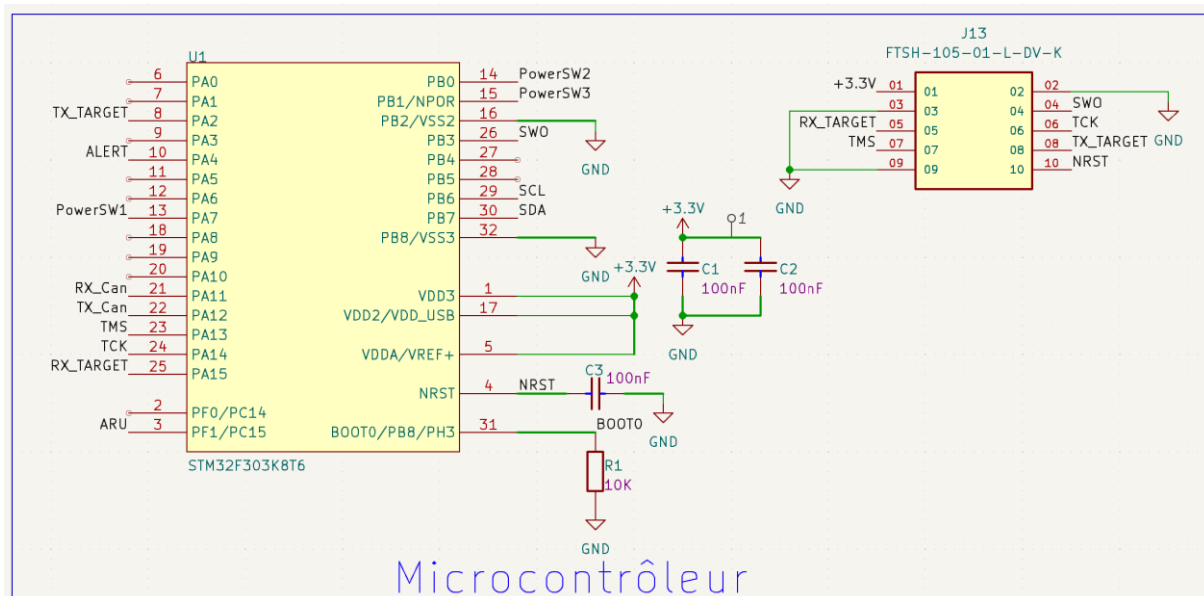


Figure 10 : Schéma du microcontrôleur et du connecteur de programmation

3.4.3. Schéma des alimentations protégé et ses circuits de mesure

La carte contient 3 sorties pour alimenter le reste du robot. Ces sorties sont protégées par le BTS441TG, un circuit de protection et interrupteur programmable, le montage est le même que celui de l'ancienne carte, et c'est le montage recommandé par le fabricant.

L'INA236 mesure la tension et le courant de l'alimentation à l'aide de sa résistance de shunt, il envoie les informations avec le bus I2C, il faut relier les signaux SDA, SCL et Alert à des résistances de pull-up. La broche AO permet de donner l'adresse du capteur, et la broche Alert permet d'alerter d'une sous/surtension sur la charge ou sur la résistance de puissance limite programmée. Le montage constructeur.

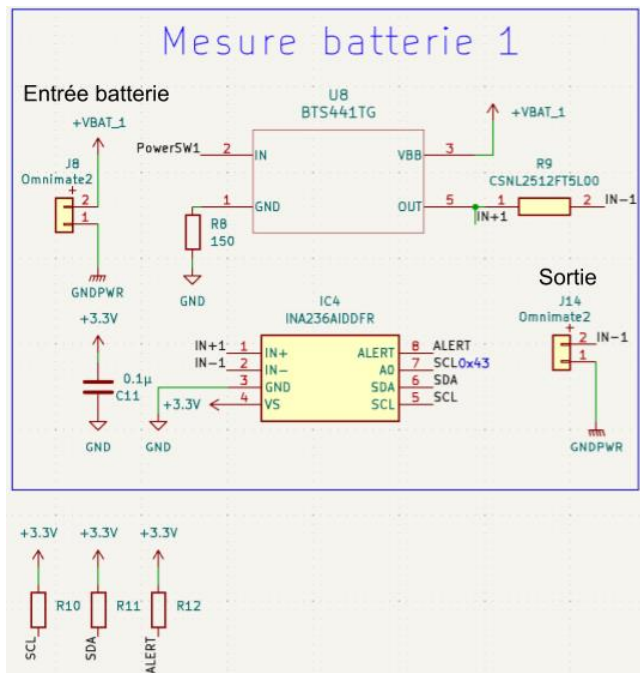


Figure 11 : Bloc d'alimentation 1 et résistances de pull-up

Tableau 2 : Adresses de l'INA236

Adresse	A0
0x40	GND
0x41	+3.3 V
0x42	SDA
0x43	SCL

3.4.4. Schéma du driver CAN

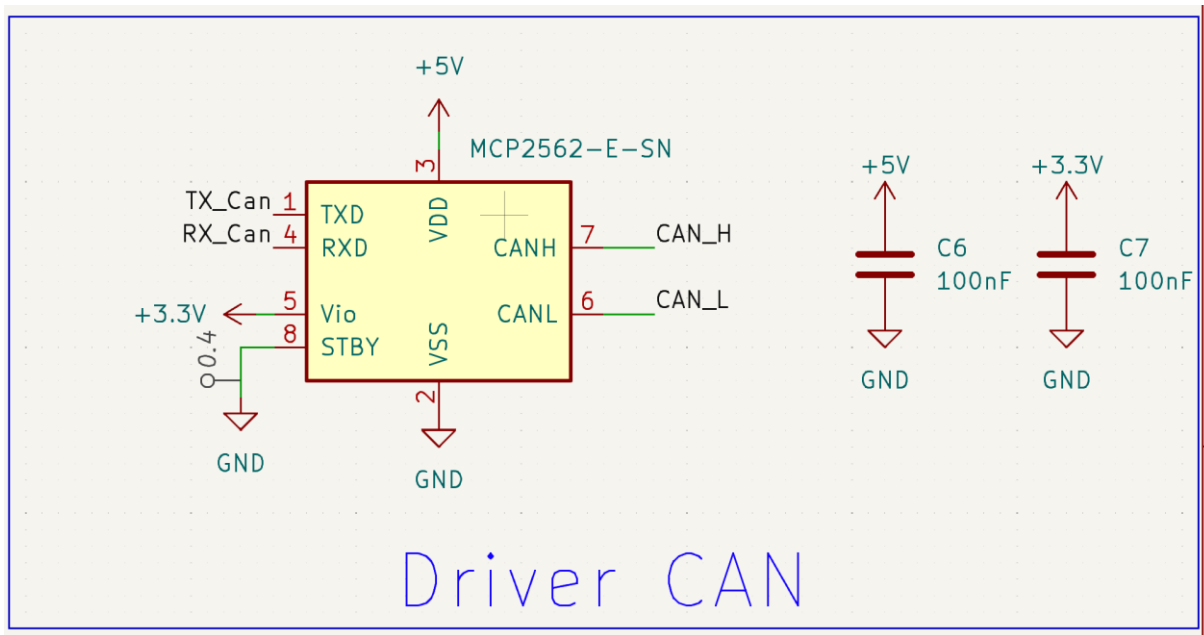


Figure 12 : Schéma du driver CAN

Le driver CAN est une interface entre bus CAN et le microcontrôleur, le câblage est celui qui est recommandé par la doc du constructeur, la broche STBY (Standby) est relié à la masse pour que le driver soit opérationnel.

3.5. Conception du PCB

Après avoir fini le schématique de la carte j'ai commencé la conception du PCB, en commençant par le placement des composants, les composants qui sont proche dans le schématique doivent être proche dans le PCB. Le placement recommandé pour l'INA236 est le suivant :

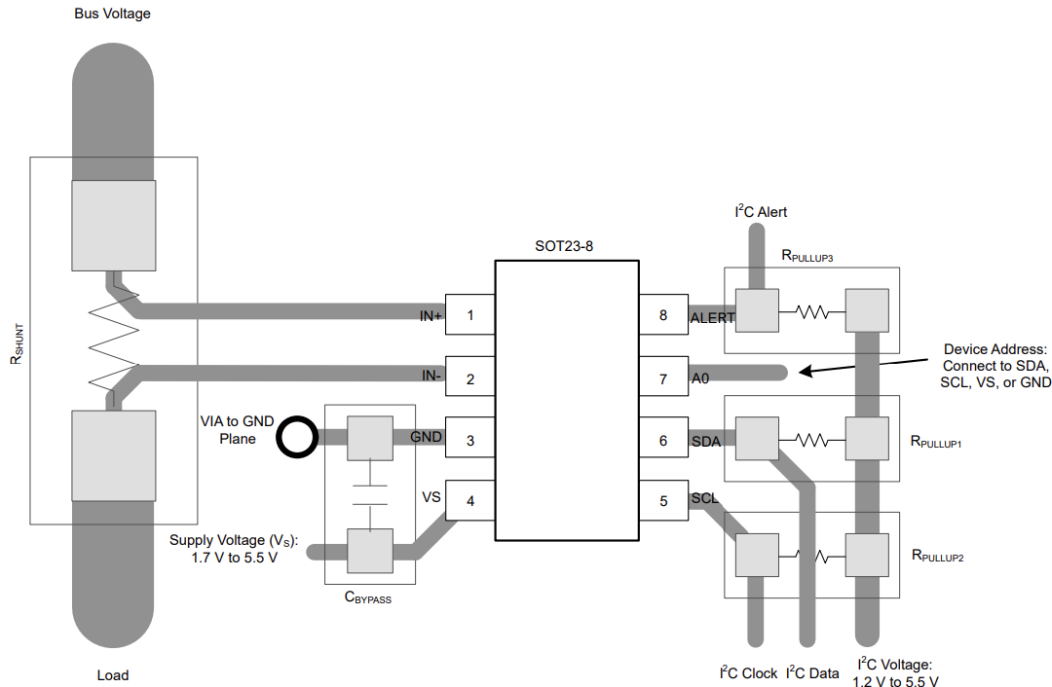


Figure 13 : Placement recommandé de l'INA236

Initialement, le placement des composants était similaire à l'ancienne carte, mais j'ai mis des composants CMS sur la face du dessous ce qui rend la soudure au four plus difficile. Le placement des composants a donc été refait par le professeur.

Ensuite j'ai routé la carte, les pistes de puissances doivent être assez épaies pour laisser passer du courant, et les pistes des signaux et de petite puissance sont moins épaies. Il faudra également éviter de router une piste de signal entre deux pistes de puissances pour éviter les problèmes d'interférences.

Un plan de masse a été ajouté à la fin sur les deux faces de cuivre et j'ai ajouté la sérigraphie pour les composants. Et enfin j'ai envoyé la carte au professeur. La carte a ensuite été soudée par M.Ardiller.

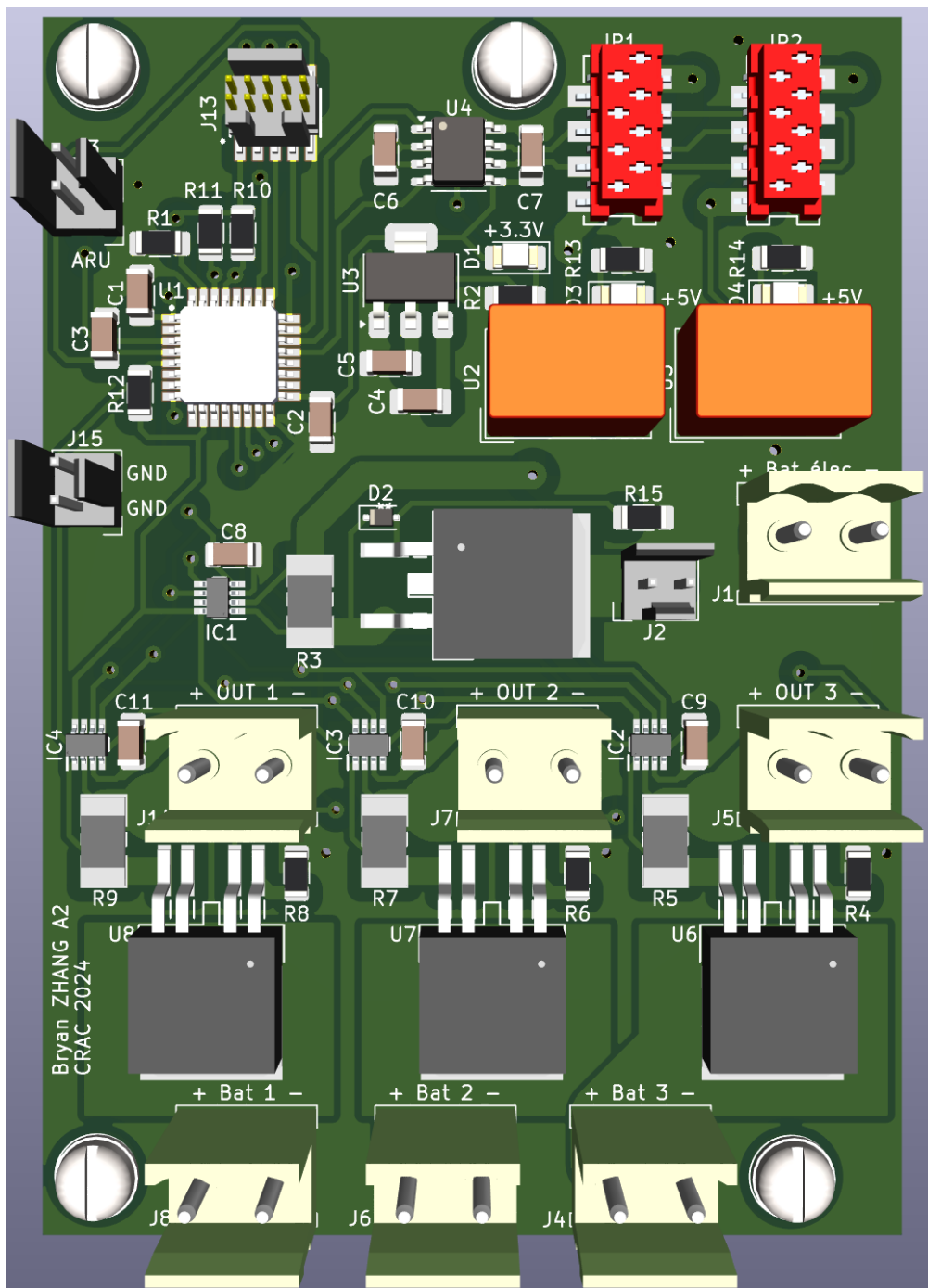


Figure 14 : Modèle 3D de la carte envoyé en production

3.6. Test et Programmation de la nouvelle carte

La plupart des tests sur la carte et le programme est le suivant :

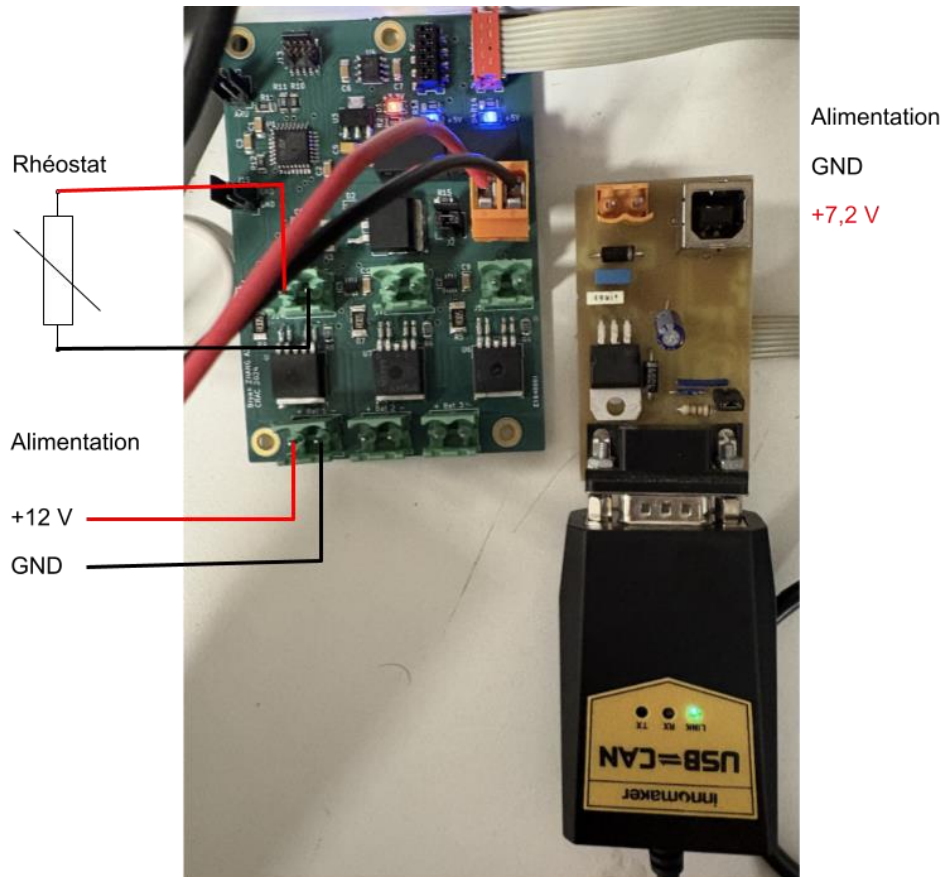


Figure 15 : Câblage du test de la carte

3.6.1. Test et programmation des capteurs

Pour les INA236 une bibliothèque toute faite a été utilisé, on peut la trouver sur platformIO ou sur GitHub (<https://github.com/RobTillaart/INA236>). On doit commencer par initialiser un objet de la classe INA236 en donnant l'adresse du capteur :

```
INA236 BatterieMain(ADRESSE_CAPTMAIN);
```

Il faut ensuite Initialiser les capteurs :

```
Wire.begin();  
// Initialisation batterie 1  
Batterie1.begin();  
Batterie1.setMaxCurrentShunt(3, 0.00471, true); // Courant max en A et  
RSHUNT en ohm, normalisation
```

Pour lire les valeurs on peut utiliser :

```
Vbatt1 = Batterie1.getBusVoltage();  
Ibatt1 = Batterie1.getCurrent_mA();
```

On peut également avoir la puissance (elle peut être différent du produit du tension et courant lu avec **getBusVoltage** et **getCurrent** respectivement). Il est possible d'obtenir les valeurs

dans d'autres rangées de précision, par exemple on peut avoir la tension en V, mV ou μV en précisant l'unité (sauf pour le V). Lors du test des voies des batteries 1-3 il faut s'assurer que les interrupteurs sont passant, pour les tests j'ai fermé le contact des interrupteurs dans le programme avec **digitalWrite(HIGH)** ;

La tension, courant et puissance sont calculé de la manière suivante (équations trouvées dans la datasheet de l'INA236) :

$$Power[W] = 32 \cdot CURRENT_{LSB} \cdot POWER$$

$$CURRENT_{LSB} = \frac{I_{max}}{2^{15}}$$

$$V_{bus} (V) = 1,6 \cdot 10^{-3} \cdot VOLTAGE \quad (\text{Calcul trouvé dans la bibliothèque})$$

$$Current[A] = CURRENT_{LSB} \cdot CURRENT$$

Pour utiliser la broche Alert il faut initialiser la broche en donnant le mode de fonctionnement et la valeur limite. Exemple pour alerter d'un dépassement de la limite de puissance :

```
pinMode(PINALERT, INPUT);
Batterie1.setAlertLimit(limit);
Batterie1.setAlertRegister(INA236_POWER_OVER_LIMIT); // masque pour le mode
de fonctionnement que l'on veut
```

Il y a 5 modes de fonctionnement pour la broche alert :

- Shunt voltage overlimit (SOL)
- Shunt voltage underlimit (SUL)
- Bus voltage overlimit (BOL)
- Bus voltage underlimit (BUL)
- Power overlimit (POL)

La variable limit est la valeur codée dans le capteur pour l'avoir pour la tension ou puissance, on peut le calculer de la manière suivante :

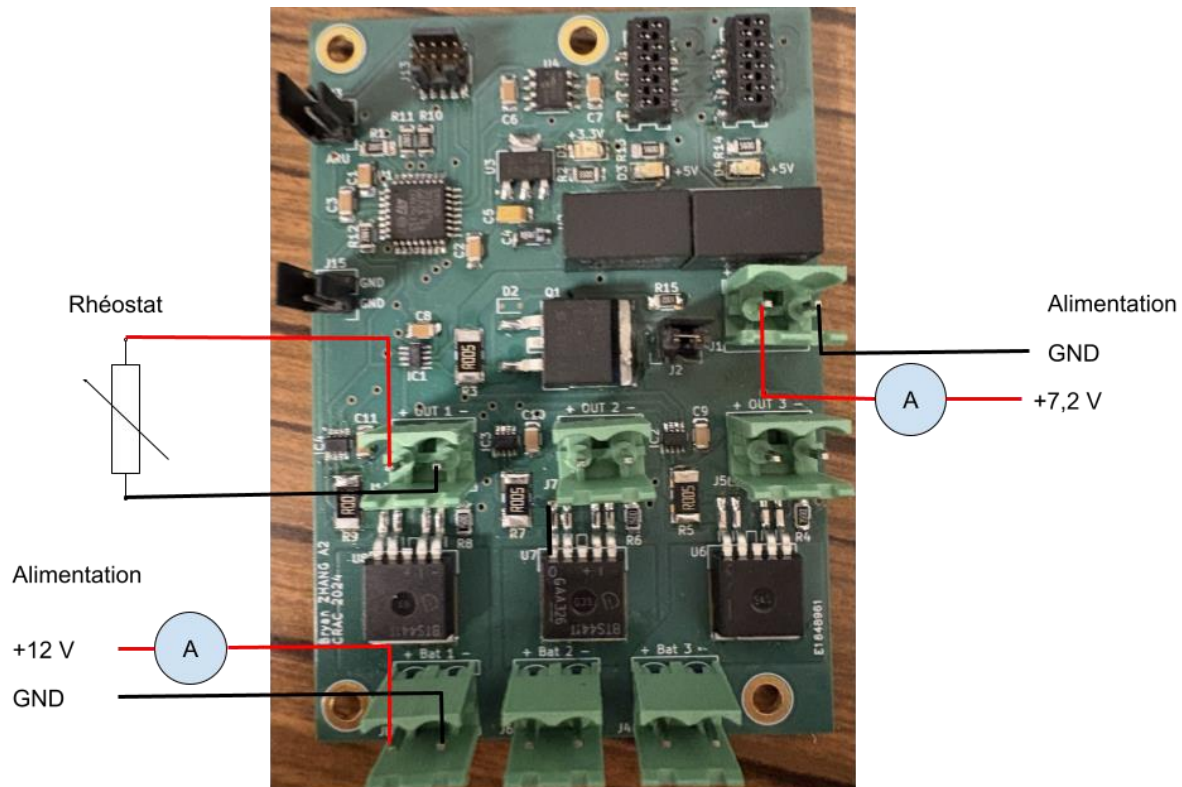
$$limit = \frac{P_{limit}}{32 \cdot CURRENT_{LSB}} \quad (\text{Power over limit})$$

$$limit = \frac{V_{lim}}{1,6 \cdot 10^{-3}} \quad (\text{over or under voltage})$$

3.6.2. Étalonnage des capteurs

Après avoir testé la lecture des courant et tension j'ai remarqué que la tension était précise (écart de quelques mV), mais pour le courant l'écart pouvait être plutôt grand (quelques %), cet écart est dû au fait que lors des test la valeur théorique du shunt de mesure a été mise dans **setMaxCurrentShunt**.

Il faut donc effectuer un étalonnage du capteur pour avoir une mesure plus précise. Le câblage est le suivant (les ampèremètres sont à placer dans les voies que l'on veut étalonner) :



J'ai relevé le tableau suivant :

Tableau 3 : Étalonnage des capteurs

	Icapteur	Imultimètre (mA)	Rth (ohm)	Rshunt (ohm)	erreur (%)
Voie 1	385	409	0,005	0,00471	5,87%
Voie 2	777	756	0,005	0,00514	-2,78%
Voie 3	727	757	0,005	0,00480	3,96%

Après l'étalonnage les capteurs ont gagné en précision (écart de quelques mA par rapport à la valeur réelle).

3.6.3. Programmation de l'interruption sur l'arrêt d'urgence

Lorsque l'interrupteur d'arrêt d'urgence est actionné, le robot est obligé de s'arrêter, pour cela il faut couper les alimentations et prévenir dans le bus CAN que l'ARU a été actionné.

Pour faire cela l'arrêt d'urgence a été initialisé dans le setup comme entrée TOR en pull-up interne (voir : [3.1. Prise en main du STM32](#))

), et j'ai configuré une interruption qui sera plus efficace que scruter la valeur de l'arrêt d'urgence. L'initialisation l'interruption sur front descendant est la suivante :

```
attachInterrupt(digitalPinToInterrupt(PINARU), arret_urgence, FALLING);
```

Le premier argument est la broche de l'arrêt d'urgence, la syntaxe utilisée est celle qui est recommandée, ensuite on met le nom de la fonction d'interruption à appeler (pas besoin de prototype de fonction) et enfin on précise le mode de fonctionnement :

- LOW : active l'interruption à l'état bas
- HIGH : active l'interruption à l'état haut
- FALLING : active sur un front descendant
- RISING : active sur un front montant
- CHANGE : active lorsque l'état de la broche change

Et ensuite il faut écrire la fonction d'interruption, elle ne retourne rien et ne prend rien en argument. Dans cette fonction on éteint toutes les alimentations et on envoie une trame sur le bus CAN pour avertir toutes les cartes que l'arrêt d'urgence a été actionné.

3.6.4. Détection d'un court-circuit

Lors du test des capteurs de courant et tension, j'ai remarqué avec mon professeur que le BTS441 se mettait en protection thermique et coupe l'alimentation lors d'un court-circuit, mais la coupure de l'alimentation s'arrêtait au bout d'un petit moment et il se remet en protection thermique.

J'ai alors décidé de programmer une détection de court-circuit, un court-circuit s'accompagne d'une grande chute de tension, alors si la tension est basse (en dessous de 4,8 V) alors que le courant est très grand (plus de 500 mA), un court-circuit est présent et la carte coupe l'alimentation et envoie une trame pour signaler le court-circuit.

3.6.5. Programmation du BUS CAN

La carte envoie en continu des trames, d'autres de manière occasionnelle, et il regarde en continu si une trame a été reçue.

Tableau 4 : Trames CAN envoyées et reçues

Envoi en continu	Envoi occasionnelle	Réception
V_I_BATTMAIN	FIN_BOOT_CARTE_PUISSANCE	CMD_INTERR_1
V_I_BATT1	CC_BATT1	CMD_INTERR_2
V_I_BATT2	CC_BATT2	CMD_INTERR_3
V_I_BATT3	CC_BATT3	STOP_ROBOT
NBRE_CELL_BATT	ARU	

Pour plus d'informations, voir le google Sheets qui contient les ID et légendes pour les trames CAN :

<https://docs.google.com/spreadsheets/d/1iT7SM82AJtVZP3JefO9gyoNWXbmPJLumuiXdtFRrQHY/edit?usp=sharing>

Envoi des tensions et courants

Pour envoyer les valeurs des tensions et courants j'ai décidé d'envoyer le code hexadécimal des tensions et courants dans le bus CAN. Pour cela j'ai créé une fonction qui met la valeur du flottant dans un tableau de 4 octets que l'on va envoyer dans le bus CAN et une autre pour récupérer la valeur codée dans un tableau. Pour ces deux fonctions, il faut au préalable avoir un tableau qui contient ou qui va contenir le code hexadécimal.

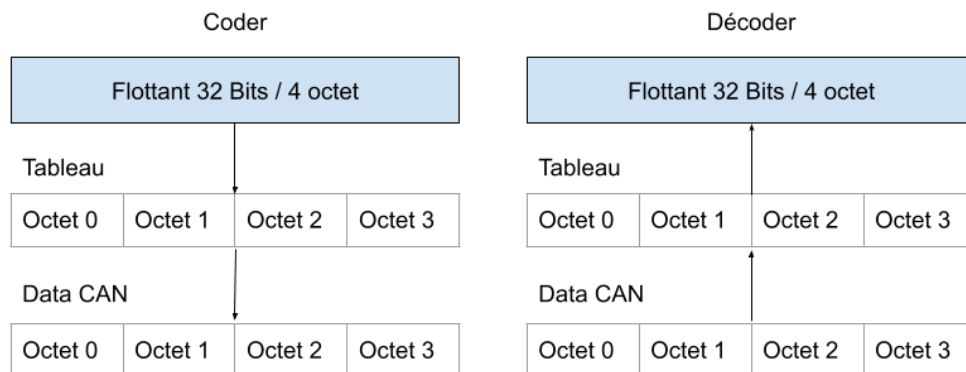


Figure 17 : Fonctionnement du codage/décodage d'un flottant dans le bus CAN

Les fonctions sont les suivantes :

```
void conversion_float_4char(float nombre, unsigned char *adresse_tableau)
{
    float *ptr;
    ptr = (float *)adresse_tableau; // donne l'adresse du tableau au pointeur
    *ptr = nombre;                  // met à l'adresse du tableau la valeur réel (code binaire)
}

float conversion_4char_float(unsigned char *adresse_tableau)
{
    float nombre, *ptr;
    ptr = (float *)adresse_tableau; // donne l'adresse du tableau au pointeur qui pointe vers un float
    nombre = *ptr;                  // met dans le nombre le contenu à l'adresse du pointeur, le nombre réel codé en binaire
    return nombre;
}
```

Les valeurs dans le tableau sont ensuite stockées dans les données à envoyer dans le bus CAN. La carte envoie les tensions et courants de toutes les alimentations et le nombre de cellules des batteries.

Filtrage des trames reçues

Avec la bibliothèque STM32CAN, il est possible de filtrer les trames CAN pour recevoir uniquement les trames que l'on veut recevoir :

```
// Filtres pour recevoir des message spécifiques
// bank number, id, masque
Can1.setFilter(0, INTERRUPTEUR_BATT1, 0x1FFFFFFF);
Can1.setFilter(1, INTERRUPTEUR_BATT2, 0x1FFFFFFF);
Can1.setFilter(2, INTERRUPTEUR_BATT3, 0x1FFFFFFF);
Can1.setFilter(3, STOP_ROBOT_FIN_MATCH, 0x1FFFFFFF);
```

Le bank number doit être différent pour chaque id.

The screenshot shows a software interface for a STM32 microcontroller. On the left, a code editor displays C++ code for initializing the CAN bus and setting filters. The code includes comments in French and function calls like `Can1.setFilter`. On the right, a control panel allows sending and receiving CAN frames. Below the control panel, a table displays the received frames, filtered by ID.

SeqID	TimeStamp	Channel	Direction	FrameId	FrameType	FrameFormat	Length
0	2328.502.540	0	Recv[0]	0x107	Data Frame	Standard Frame	8
1	2333.596.538	0	Send[0]	0x613	Data Frame	Standard Frame	8
2	2341.517.793	0	Send[1]	0x100	Data Frame	Standard Frame	8
3	2346.067.833	0	Send[2]	0x110	Data Frame	Standard Frame	8
4	2385.259.978	0	Send[3]	0x10	Data Frame	Standard Frame	8
5	2390.108.730	0	Send[4]	0x40	Data Frame	Standard Frame	8
6	2394.346.873	0	Send[5]	0x140	Data Frame	Standard Frame	8

At the bottom of the interface, a terminal window shows the raw CAN data as hexadecimal strings, such as `msg : 0 0 0 ff dd 44 0 0`.

Figure 18 : Capture d'écran montrant le filtrage des ID

On peut voir qu'uniquement les trames des ID que l'on veut a été reçu soit 0x100, 0x110, 0x10 et 0x40.

Traitement des Trames CAN reçu

Lorsque la carte reçoit une trame, il la traite. L'algorithme est le suivant :

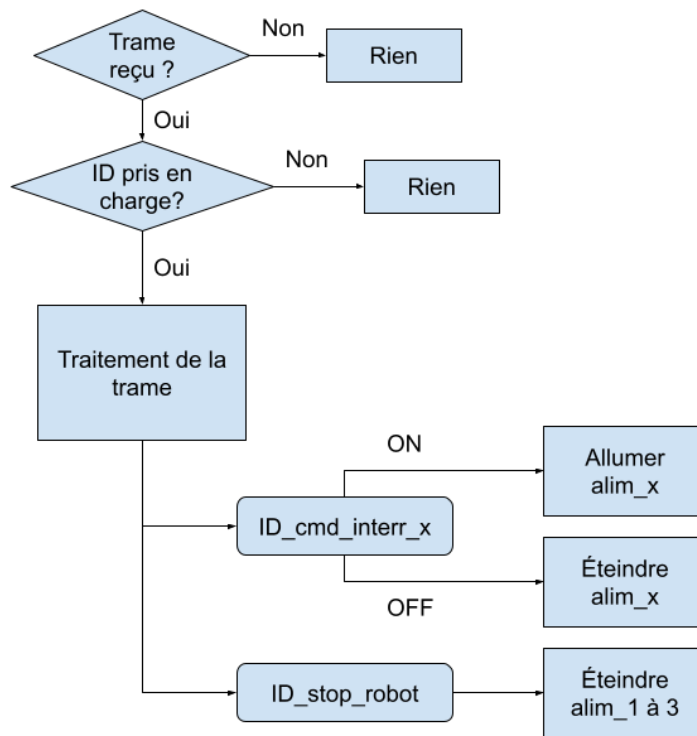


Figure 19 : Traitement des trames CAN reçu

On peut commander les interrupteurs depuis le bus CAN en envoyant un 1 ou un 0 sur la trame pour allumer ou éteindre l'alimentation. Une autre trame ordonne au robot de s'arrêter si la carte reçoit la trame il éteint toutes les alimentations.

4. Résultats obtenus

À la fin du projet j'ai réussi à avoir une carte fonctionnelle, les circuits de protections fonctionnent, les mesures sont précises et la communication avec le bus CAN fonctionne. Dans le moniteur série on a l'affichage suivant :

```

aru = 1 | alert = 1 | VbattMAIN:7.25V | IbattMAIN:32.60mA | Vbatt1:0.00V | Ibatt1:-1.30mA | Vbatt2:0.00V | Ibatt2:0.10mA | Vbatt3:0.00V | Ibatt3:-0.30mA
aru = 1 | alert = 1 | VbattMAIN:7.25V | IbattMAIN:33.80mA | Vbatt1:0.01V | Ibatt1:-1.50mA | Vbatt2:0.01V | Ibatt2:-0.50mA | Vbatt3:0.01V | Ibatt3:-0.80mA
aru = 1 | alert = 1 | VbattMAIN:7.26V | IbattMAIN:32.60mA | Vbatt1:0.00V | Ibatt1:-1.30mA | Vbatt2:0.00V | Ibatt2:-0.10mA | Vbatt3:0.00V | Ibatt3:-0.80mA
aru = 1 | alert = 1 | VbattMAIN:7.25V | IbattMAIN:32.70mA | Vbatt1:0.00V | Ibatt1:-0.30mA | Vbatt2:0.00V | Ibatt2:0.00mA | Vbatt3:0.00V | Ibatt3:-1.00mA
aru = 1 | alert = 1 | VbattMAIN:7.26V | IbattMAIN:33.00mA | Vbatt1:0.00V | Ibatt1:-0.70mA | Vbatt2:0.00V | Ibatt2:-0.00mA | Vbatt3:0.01V | Ibatt3:-0.40mA
aru = 1 | alert = 1 | VbattMAIN:7.26V | IbattMAIN:32.70mA | Vbatt1:0.00V | Ibatt1:-0.80mA | Vbatt2:0.00V | Ibatt2:-0.50mA | Vbatt3:0.00V | Ibatt3:-0.50mA
aru = 1 | alert = 1 | VbattMAIN:7.26V | IbattMAIN:32.30mA | Vbatt1:0.01V | Ibatt1:-0.80mA | Vbatt2:0.01V | Ibatt2:-0.40mA | Vbatt3:0.01V | Ibatt3:-0.70mA
aru = 1 | alert = 1 | VbattMAIN:7.25V | IbattMAIN:32.10mA | Vbatt1:0.00V | Ibatt1:-0.10mA | Vbatt2:0.00V | Ibatt2:0.10mA | Vbatt3:0.00V | Ibatt3:-0.70mA
aru = 1 | alert = 1 | VbattMAIN:7.25V | IbattMAIN:32.60mA | Vbatt1:0.00V | Ibatt1:-1.20mA | Vbatt2:0.00V | Ibatt2:-0.20mA | Vbatt3:0.00V | Ibatt3:-1.20mA
aru = 1 | alert = 1 | VbattMAIN:7.25V | IbattMAIN:31.80mA | Vbatt1:0.00V | Ibatt1:-0.70mA | Vbatt2:0.00V | Ibatt2:-0.50mA | Vbatt3:0.00V | Ibatt3:-0.70mA
aru = 1 | alert = 1 | VbattMAIN:7.26V | IbattMAIN:33.00mA | Vbatt1:0.00V | Ibatt1:-0.80mA | Vbatt2:0.00V | Ibatt2:-0.90mA | Vbatt3:0.00V | Ibatt3:-0.40mA

```

Figure 20 : Affichage des valeurs lue par les capteurs

La carte est également plus compacte que l'ancienne.

Tableau 5 : Comparaison de la taille de la carte industrialisée contre l'ancienne carte

	Ancienne carte	Carte Industrialisé	Comparaison
Longueur (mm)	98	88	10,20%
Largueur (mm)	70	62	11,43%
Épaisseur (mm)	30	15	50,00%

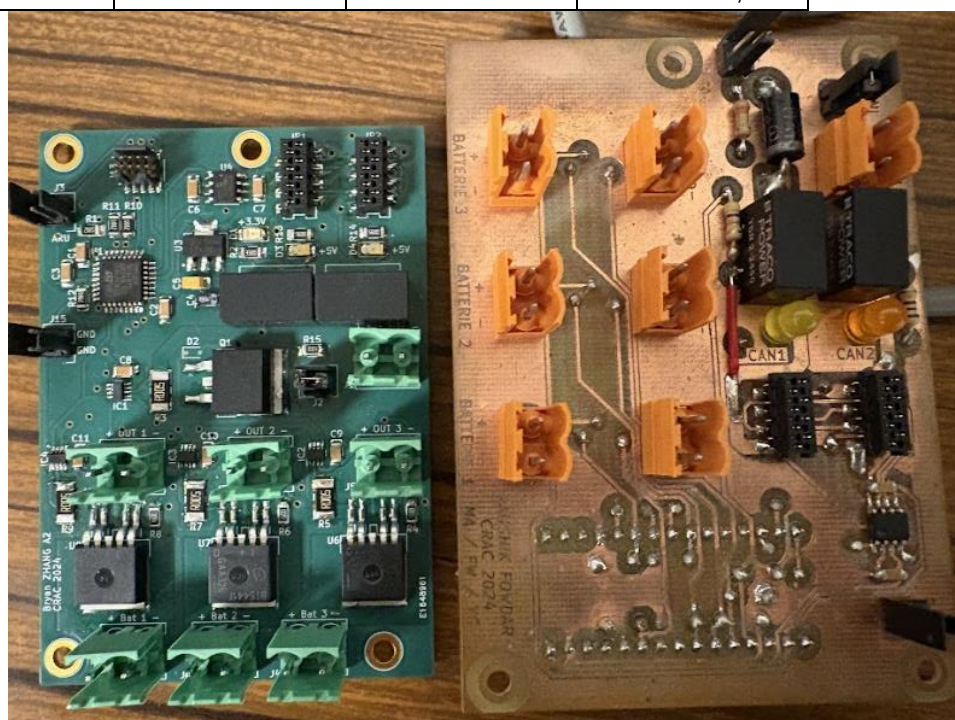


Figure 21 : Carte industrialisé à côté de l'ancienne carte

Conclusion

Pour conclure ce projet à été très instructif, il m'a permis de mobiliser mes connaissances acquises pendant les cours, en effet j'ai utilisé pour la première fois des composants cms sur une carte, j'ai pu découvrir comment fonctionne des circuits de protections, j'ai appris comment fonctionne un bus CAN et comment l'utiliser pour communiquer avec d'autres appareils. J'ai aussi pu programmer la carte en C++ sur Arduino avec PlatformIO. J'ai pu développer mes connaissances en électronique et en programmation. Et enfin le projet est terminé.

Quelques problèmes ont été rencontrés notamment pour la prise en main du microcontrôleur ou je me suis aperçu que seul le protocole d'upload mbed fonctionne, que les broches PA5 et PA6 ne sont utilisable qu'en entrée analogique sinon d'autres broches seront programmées, l'inversion de la sérigraphie par rapport au nom des net pour les entrées de batteries et les Traco TSR-2-2450 ont la mauvaise empreinte.

Annexes

Annexe 1 : Doc des composants

INA236 :

<https://www.ti.com/lit/gpn/INA236>

BTS441TG :

https://www.mouser.fr/ProductDetail/Infineon-Technologies/BTS441TG?qs=mzcOS1kGbgf58yqzVPhgKw%3D%3D&srsId=AfmBOoof7ILLIvdFews9Ui-TV7kRIltus_dFHs7H9PxvPE4z6ii175cS

SQM40041EL_GE3 :

https://www.mouser.com/ProductDetail/Vishay-Semiconductors/SQM40031EL_GE3?qs=4pziT8jjbhe%2FuOSqrrXd%2FQ%3D%3D&srsId=AfmBOorcutJoVHt9234PxnucKK7wfGUVZmm_WiveQQ7_vwse2NLPuCLW *

TSR-2-2450 :

https://www.mouser.fr/ProductDetail/TRACO-Power/TSR-2-2450?qs=NtE2QagKf6RzknAolz%2FTQ%3D%3D&srsId=AfmBOoqghp8_d6X7JJ0KEitOKKuZ0paKDvjJm18iS6f8iwuiOql6-8FU

AZ1117-3.3 :

https://www.mouser.fr/ProductDetail/Diodes-Incorporated/AZ1117D-3.3TRE1?qs=FKu9oBikfSmTTprBI2eFQQ%3D%3D&srsId=AfmBOooGF6-uWY03gJLwJOxmYdP4DpqYno_mplhSBAQymCigAUtSgXsq

MCP2562-E/SN

https://www.mouser.fr/ProductDetail/Microchip-Technology/MCP2562-E-SN?qs=6qKXPk0vzx3LIPUMCCw%252BCg%3D%3D&srsId=AfmBOopGM8Eqcxpc0_fSzPhK_moUt_6e17JKWNoCLJCL1gMkj0rN_48

Annexe 2 : manuel d'utilisateur de l'USB2CAN

<https://github.com/INNO-MAKER/usb2can/blob/master/Document%EF%BC%88read%20me%20first%EF%BC%89/USB2CAN%20UserManual%20v.1.8.pdf>

Annexe 3 : Projet KiCad de la carte puissance (Avec la correction de sérigraphie et empreinte)

https://drive.google.com/file/d/13JI4LmcO5JpdWw6Ys9MJsPXT3uxtk39C/view?usp=drive_link

Annexe 4 : Bibliothèques

https://github.com/pazi88/STM32_CAN

<https://github.com/RobTillaart/INA236>

Annexe 5 : ID des bus CAN

<https://docs.google.com/spreadsheets/d/1iT7SM82AJtVZP3JefO9gyoNWxbmPJLumuiXdtFRrQHY/edit?usp=sharing>

Annexe 6 : Programme de la carte

https://github.com/Z-bryan/Prog_Carte_puissance_crac